Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

ОТЧЕТ
по лабораторной работе № 1
на тему

ОСНОВЫ PYTHON

Вариант 4

Студент Боб М.В.

Руководитель Лебедевич А.В.

Минск 2026

1 ЦЕЛЬ ВЫПОЛНЕНИЯ ЛАБОРАТОРНОЙ РАБОТЫ

Целью лабораторной работы является изучение основных конструкций языка программирования Python и приобретение практических навыков их применения: работа с числами, строками, списками, кортежами и словарями; создание функций; обработка исключений с помощью конструкции try/except/finally; создание классов, использование инкапсуляции, наследования и полиморфизма, а также методов экземпляра, статических методов и методов класса.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Ввод-вывод, строки и циклы

Функция input() считывает строку с клавиатуры и всегда возвращает значение типа str, поэтому для получения числа результат приводят к нужному типу функциями int() или float(). Вывод выполняется функцией print(); для удобного форматирования применяются f-строки, например f"Сумма: {s}".

Строка – неизменяемая последовательность символов. Её можно обходить циклом for, получать отдельные символы по индексу и применять срезы вида s[start:stop:step]: срез s[::-1] возвращает строку в обратном порядке, что позволяет проверить, является ли слово палиндромом. Метод split() разбивает строку на слова, strip() удаляет символы по краям, lower() приводит к нижнему регистру, isdigit() проверяет, состоит ли строка только из цифр. Цикл while выполняется, пока условие истинно, оператор break досрочно завершает цикл, а continue переходит к следующей итерации.

2.2 Списки и кортежи

Список (list) – изменяемая упорядоченная коллекция элементов, доступ к которым осуществляется по индексу, начиная с нуля. Метод append() добавляет элемент в конец, remove() удаляет первое вхождение значения, copy() создаёт копию списка. Встроенные функции len(), sum(), max(), min() возвращают длину, сумму, наибольший и наименьший элементы, а sorted() возвращает новый отсортированный список (параметр reverse=True задаёт сортировку по убыванию). Срезы работают так же, как для строк: lst[1::2] выбирает элементы с нечётными индексами.

Генератор списка позволяет построить список одним выражением: [x * 2 for x in data if x > 0].

Кортеж (tuple) – неизменяемая упорядоченная последовательность. После создания элементы кортежа нельзя изменить, добавить или удалить, поэтому кортеж формируется сразу целиком, например:

```
tuple(random.randint(1, 100) for _ in range(10))
```

Модуль random предоставляет функции генерации случайных чисел.

2.3 Словари

Словарь (dict) хранит пары «ключ – значение». Ключи уникальны и должны быть неизменяемыми, значения могут быть любого типа, в том числе списками. Доступ к значению осуществляется по ключу (d[key]), наличие ключа проверяется оператором in, метод get(key, default) возвращает значение по умолчанию при отсутствии ключа. Методы items(), keys(), values() возвращают пары, ключи и значения соответственно.

Функция zip() объединяет несколько последовательностей в пары соответствующих элементов, поэтому словарь из двух списков одинаковой длины строится выражением dict(zip(keys, values)). Выражение min(d, key=d.get) возвращает ключ с минимальным значением.

2.4 Функции

Функция – именованный блок кода, который может принимать аргументы и возвращать результат с помощью оператора return. Функция объявляется ключевым словом def; если return не указан, функция возвращает None. Использование функций позволяет разбивать программу на логические части и повторно использовать код.

Для проверки типа переданного аргумента используется функция isinstance(obj, type), что позволяет реализовать разное поведение одной функции для разных типов данных (словаря, списка, числа, строки). Параметр key функций min(), max() и sorted() принимает функцию, по результату которой сравниваются элементы.

2.5 Обработка исключений

Исключение – объект, описывающий ошибку, возникшую во время выполнения программы. Часто встречаются исключения ValueError (неверное значение), TypeError (неверный тип) и ZeroDivisionError (деление на ноль). Для

обработки исключений применяется конструкция try/except/else/finally. В блок try помещается код, способный вызвать ошибку; блоки except перехватывают исключения указанных типов, причём тип можно указать вместе с переменной, содержащей описание ошибки (except ValueError as e). Необязательный блок else выполняется, если исключений не возникло. Блок finally выполняется всегда, независимо от того, произошла ошибка или нет, и обычно используется для освобождения ресурсов. Оператор raise позволяет сгенерировать исключение вручную.

2.6 Классы и объекты

Класс – шаблон, описывающий атрибуты (данные) и методы (действия) объектов; объект – конкретный экземпляр класса. Класс объявляется ключевым словом class, метод __init__ является конструктором и вызывается при создании объекта, первый параметр методов self ссылается на текущий экземпляр. Метод __str__ определяет строковое представление объекта при выводе функцией print().

Инкапсуляция – сокрытие внутреннего устройства объекта. В Python атрибут с двумя нижними подчёркиваниями (self.__name) считается приватным; доступ к нему организуется через методы доступа (геттеры, get_name()) и изменения (сеттеры, set_name()), которые могут проверять корректность присваиваемых значений.

Наследование позволяет создать новый класс на основе существующего (class Pen(Stationery)): дочерний класс получает все атрибуты и методы родителя и может переопределять их. Функция super() даёт доступ к методам родительского класса. Полиморфизм – возможность вызывать один и тот же метод у объектов разных классов, при этом выполняется реализация, соответствующая классу конкретного объекта.

2.7 Методы экземпляра, статические методы и методы класса

Метод экземпляра принимает первым параметром self и работает с данными конкретного объекта. Статический метод объявляется декоратором @staticmethod, не получает ни объекта, ни класса и представляет собой обычную функцию, логически относящуюся к классу. Метод класса объявляется декоратором @classmethod и первым параметром получает сам класс cls; такие методы работают с атрибутами класса, общими для всех экземпляров, и часто используются как альтернативные конструкторы.

3 ФОРМУЛИРОВКА ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

Индивидуальное задание (вариант 4) состоит из трёх частей.

Часть 1. Встроенные типы данных

1. Вычислить сумму цифр введённого натурального числа.

2. Ввести строку текста. Вывести на экран все слова-палиндромы.

3. Найти произведение элементов списка с нечётными номерами. Найти наибольший элемент списка, затем удалить его и вывести новый список. Вывести на экран три наибольших элемента.

4. Даны два списка одинаковой длины. Создать из них словарь таким образом, чтобы элементы первого списка были ключами, а элементы второго – соответственно значениями словаря.

5. Реализовать программу «Ювелирный магазин» с шестью пунктами меню. Имеется словарь, где ключ – название изделия, а значение – список, содержащий состав изделия (золото, серебро и т. п.), цену и количество (шт.), которое есть в магазине. Пункты меню: 1) просмотр описания (название – описание); 2) просмотр цены (название – цена); 3) просмотр количества (название – количество); 4) вся информация; 5) покупка (с клавиатуры вводятся название изделия и его количество, n – выход; посчитать цену выбранных товаров и сколько товаров осталось в изначальном списке); 6) до свидания.

6. Создать кортеж из 10 случайных чисел. Найти его максимальный и минимальный элементы.

Часть 2. Функции и обработка исключений

1. Написать функцию prime, принимающую один аргумент – номер простого числа в последовательности простых чисел, и возвращающую само простое число, находящееся под этим номером.

2. Написать функцию, принимающую один аргумент. Если передаётся словарь – вывести ключ с минимальным значением. Если список – найти произведение между первым и вторым нулевыми элементами, удалить все повторяющиеся элементы. Если число – вывести все делители данного числа.

Если строка – проверить, является ли она палиндромом, определить количество гласных и согласных. Выполнить проверку для всех перечисленных случаев.

3. Найти сумму каждой строки матрицы размером m × n.

4. Написать программу, демонстрирующую работу конструкции try/except/finally.

Часть 3. Классы и объекты

1. Создать класс Square с тремя методами: нахождение периметра, площади и диагонали квадрата.

2. Реализовать класс Stationery (канцелярская принадлежность) с атрибутом title (название) и методом draw (отрисовка), выводящим сообщение «Запуск отрисовки». Создать три дочерних класса Pen (ручка), Pencil (карандаш), Handle (маркер), в каждом переопределить метод draw так, чтобы он выводил уникальное сообщение. Создать экземпляры классов и проверить, что выведет метод для каждого экземпляра.

3. Разработать класс Faculty, включающий название факультета. Реализовать класс Student, включающий Ф. И. О. студента, год рождения и результаты сдачи последней сессии. Классы должны содержать методы доступа и изменения всех полей. Написать программу, которая считывает данные о студентах и выдаёт информацию об их успеваемости.

4. Придумать класс самостоятельно, реализовать в нём методы экземпляра класса, статические методы и методы класса.

4 ПРОГРАММНЫЙ КОД РЕШЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

Решение каждой части оформлено отдельным файлом (part1.py, part2.py, part3.py). Каждое задание реализовано отдельной функцией taskN(), выбор задания выполняется через меню в функции main().

4.1 Часть 1. Встроенные типы данных

Задание 1. Сумма цифр натурального числа

```
import random

def task1():
    text = input("Введите натуральное число: ").strip()

    # Натуральное число: только цифры и больше нуля.
    # str.isdigit() вернёт True, только если все символы строки цифры.
    if not text.isdigit() or int(text) == 0:
        print("Ошибка: нужно натуральное число (1, 2, 3, ...)")
        return

    # Способ 1: строка это последовательность символов, проходим по ней
    # и каждую цифру-символ превращаем в число.
    total = sum(int(ch) for ch in text)

    # Способ 2 (арифметический): % 10 даёт последнюю цифру, // 10 её отрезает.
    n, total2 = int(text), 0
    while n > 0:
        total2 += n % 10
        n //= 10

    print(f"Сумма цифр числа {text} = {total}")
    print(f"(проверка арифметическим способом: {total2})")
```

Задание 2. Слова-палиндромы

```
def task2():
    text = input("Введите строку текста: ")

    palindromes = []
    for word in text.split():                        # делим строку на слова по
пробелам
        clean = word.strip(".,!?;:\"'()-«»…").lower()  # убираем знаки
препинания и регистр
        # Палиндром читается одинаково в обе стороны.
        # clean[::-1] это срез с шагом -1, то есть строка задом наперёд.
        # Однобуквенные слова («а», «и») палиндромы тривиально, их не считаем.
        if len(clean) > 1 and clean == clean[::-1]:
            palindromes.append(clean)

    if palindromes:
        print("Слова-палиндромы:", ", ".join(palindromes))
    else:
```

```
        print("Палиндромов в строке нет")
```

Задание 3. Операции со списком

```
def read_list():
    """Считывает список чисел через пробел. Пустой ввод: случайный список."""
    raw = input("Введите числа через пробел (Enter: случайный список):
").strip()
    if not raw:
        nums = [random.randint(-10, 50) for _ in range(10)]
        print("Сгенерирован список:", nums)
        return nums
    return [int(x) for x in raw.split()]


def task3():
    try:
        nums = read_list()
    except ValueError:
        print("Ошибка: вводите только целые числа")
        return
    if len(nums) < 3:
        print("Нужно хотя бы 3 элемента")
        return


    # а) Произведение элементов с нечётными номерами.
    # В Python нумерация (индексы) идёт с 0, поэтому нечётные номера это 1, 3,
5, ...
    # Срез nums[1::2]: начинаем с индекса 1 и берём каждый второй.
    odd = nums[1::2]
    product = 1
    for x in odd:
        product *= x
    print(f"Элементы с нечётными индексами: {odd}")
    print(f"Их произведение: {product}")


    # б) Наибольший элемент, удаляем его и выводим новый список.
    original = nums.copy()      # копия, чтобы исходный список остался для
пункта в)
    biggest = max(nums)
    nums.remove(biggest)        # remove удаляет ПЕРВОЕ вхождение значения
    print(f"Наибольший элемент: {biggest}")
    print(f"Список после удаления: {nums}")


    # в) Три наибольших элемента исходного списка.
    # sorted(..., reverse=True) сортирует по убыванию, срез [:3] берёт первые
три.
    top3 = sorted(original, reverse=True)[:3]
    print(f"Три наибольших элемента: {top3}")
```

Задание 4. Словарь из двух списков

```
def task4():
    keys = input("Первый список (ключи) через пробел: ").split()
    values = input("Второй список (значения) через пробел: ").split()


    if len(keys) != len(values):
        print(f"Ошибка: длины списков разные ({len(keys)} и {len(values)})")
        return


    # Способ 1: вручную, по индексам.
    result = {}
```

```
    for i in range(len(keys)):
        result[keys[i]] = values[i]

    # Способ 2: zip склеивает списки в пары (ключ, значение), dict делает из пар
словарь.
    result2 = dict(zip(keys, values))

    print("Словарь:", result)
    print("То же через dict(zip(...)):", result2)
    if len(result) < len(keys):
        print("Внимание: в первом списке были повторы, у словаря ключи
уникальны,"
              " поэтому остались последние значения")
```

Задание 5. Программа «Ювелирный магазин»

```
SHOP = {
    "кольцо":   ["золото 585", 450.0, 10],
    "серьги":   ["серебро 925, фианиты", 120.0, 15],
    "цепочка":  ["золото 750", 890.0, 5],
    "браслет":  ["серебро 925", 95.5, 8],
    "кулон":    ["платина, бриллиант", 1500.0, 3],
}

DESCRIPTION, PRICE, COUNT = 0, 1, 2   # индексы внутри списка-значения, чтобы
код читался

def buy(shop):
    """Пункт «Покупка»: вводим название и количество, n: выход. Возвращает
сумму."""
    total = 0.0
    cart = {}
    while True:
        name = input("Название изделия (n: закончить покупку):
").strip().lower()
        if name == "n":
            break
        if name not in shop:
            print("Такого изделия нет. В наличии:", ", ".join(shop))
            continue
        try:
            qty = int(input("Количество, шт: "))
        except ValueError:
            print("Количество должно быть целым числом")
            continue
        if qty <= 0:
            print("Количество должно быть больше нуля")
            continue
        if qty > shop[name][COUNT]:
            print(f"В наличии только {shop[name][COUNT]} шт")
            continue

        shop[name][COUNT] -= qty                    # уменьшаем остаток в
магазине
        cost = qty * shop[name][PRICE]
        total += cost
        cart[name] = cart.get(name, 0) + qty        # get возвращает 0, если
ключа ещё нет
        print(f"Добавлено: {name} x {qty} = {cost:.2f} руб.")

    if cart:
```

```
        print("\nВаша покупка:")
        for name, qty in cart.items():
            print(f"  {name}: {qty} шт")
        print(f"Итого к оплате: {total:.2f} руб.")
    print("\nОстаток товаров в магазине:")
    for name, info in shop.items():
        print(f"  {name}: {info[COUNT]} шт")
    return total


def task5():
    shop = {k: v.copy() for k, v in SHOP.items()}   # копия, чтобы исходный
словарь не портился
    menu = ("\n=== Ювелирный магазин ===\n"
            "1. Просмотр описания\n2. Просмотр цены\n3. Просмотр количества\n"
            "4. Вся информация\n5. Покупка\n6. До свидания")
    while True:
        print(menu)
        choice = input("Ваш выбор: ").strip()
        if choice == "1":
            for name, info in shop.items():
                print(f"{name} – {info[DESCRIPTION]}")
        elif choice == "2":
            for name, info in shop.items():
                print(f"{name} – {info[PRICE]:.2f} руб.")
        elif choice == "3":
            for name, info in shop.items():
                print(f"{name} – {info[COUNT]} шт")
        elif choice == "4":
            for name, (desc, price, count) in shop.items():   # распаковка
списка в три переменные
                print(f"{name}: состав – {desc}; цена – {price:.2f} руб.; в
наличии – {count} шт")
        elif choice == "5":
            buy(shop)
        elif choice == "6":
            print("До свидания!")
            break
        else:
            print("Нет такого пункта, введите число от 1 до 6")
```

Задание 6. Кортеж из 10 случайных чисел

```
def task6():
    # Кортеж неизменяем, поэтому создаём его сразу целиком из генератора.
    numbers = tuple(random.randint(1, 100) for _ in range(10))
    print("Кортеж:", numbers)
    print("Максимальный элемент:", max(numbers))
    print("Минимальный элемент:", min(numbers))
```

Меню выбора задания

```
TASKS = {
    "1": ("Сумма цифр натурального числа", task1),
    "2": ("Слова-палиндромы", task2),
    "3": ("Операции со списком", task3),
    "4": ("Словарь из двух списков", task4),
    "5": ("Ювелирный магазин", task5),
    "6": ("Кортеж из 10 случайных чисел", task6),
}


def main():
    while True:
```

```
        print("\n===== ЛР1, часть 1, вариант 4 =====")
        for key, (title, _) in TASKS.items():
            print(f"{key}. {title}")
        print("0. Выход")
        choice = input("Номер задания: ").strip()
        if choice == "0":
            break
        if choice in TASKS:
            print(f"\n--- Задание {choice}: {TASKS[choice][0]} ---")
            TASKS[choice][1]()
        else:
            print("Нет такого задания")

if __name__ == "__main__":
    try:
        main()
    except (EOFError, KeyboardInterrupt):
        print("\nВыход")
```

4.2 Часть 2. Функции и обработка исключений

Задание 1. Функция prime

```
import random

def is_prime(x):
    """True, если x простое. Делители достаточно искать до корня из x:
    если x = a * b и a > sqrt(x), то b < sqrt(x), и мы бы уже нашли b."""
    if x < 2:
        return False
    d = 2
    while d * d <= x:
        if x % d == 0:
            return False
        d += 1
    return True


def prime(n):
    """Возвращает простое число под номером n (нумерация с 1: prime(1) = 2)."""
    if not isinstance(n, int) or n < 1:
        raise ValueError("номер должен быть натуральным числом")
    count = 0          # сколько простых уже нашли
    candidate = 1
    while count < n:
        candidate += 1
        if is_prime(candidate):
            count += 1
    return candidate


def task1():
    try:
        n = int(input("Номер простого числа: "))
        print(f"Простое число №{n}: {prime(n)}")
    except ValueError as e:
        print("Ошибка:", e)
    print("Первые 10 простых:", [prime(i) for i in range(1, 11)])
```

Задание 2. Функция, обрабатывающая аргумент разных типов

```
VOWELS = set("аеёиоуыэюяaeiouy")
```

```
def analyze(value):
    """Определяет тип аргумента через isinstance и выполняет нужную
обработку."""

    # bool это подкласс int в Python (True == 1), поэтому отсекаем его отдельно
    if isinstance(value, bool):
        print("bool не обрабатывается")

    # ---- словарь: ключ с минимальным значением ----
    elif isinstance(value, dict):
        print("Тип: словарь", value)
        if not value:
            print("Словарь пуст")
            return
        # min проходит по ключам, key=value.get говорит сравнивать их по
значениям
        key = min(value, key=value.get)
        print(f"Ключ с минимальным значением: {key!r} (значение {value[key]})")

    # ---- список: произведение между двумя первыми нулями + удаление повторов -
---
    elif isinstance(value, list):
        print("Тип: список", value)
        zeros = [i for i, x in enumerate(value) if x == 0]   # индексы всех
нулей
        if len(zeros) < 2:
            print("В списке меньше двух нулей, произведение посчитать нельзя")
        else:
            between = value[zeros[0] + 1: zeros[1]]           # срез строго
между нулями
            if not between:
                print("Между первым и вторым нулём нет элементов")
            else:
                product = 1
                for x in between:
                    product *= x
                print(f"Элементы между первым и вторым нулём: {between}")
                print(f"Их произведение: {product}")
        # Удаляем повторы с сохранением порядка. set() тоже убирает повторы,
        # но порядок элементов в множестве не гарантирован.
        unique = list(dict.fromkeys(value))
        print("Список без повторов:", unique)

    # ---- число: все делители ----
    elif isinstance(value, int):
        print("Тип: целое число", value)
        if value == 0:
            print("У нуля бесконечно много делителей")
            return
        n = abs(value)
        divisors = [d for d in range(1, n + 1) if n % d == 0]
        print(f"Делители числа {value}: {divisors}")

    # ---- строка: палиндром? гласные и согласные ----
    elif isinstance(value, str):
        print("Тип: строка", repr(value))
        letters = [ch.lower() for ch in value if ch.isalpha()]   # только буквы,
без пробелов
        clean = "".join(letters)
        if clean and clean == clean[::-1]:
```

```
            print("Строка является палиндромом (без учёта пробелов и регистра)")
        else:
            print("Строка не палиндром")
        vowels = sum(1 for ch in letters if ch in VOWELS)
        print(f"Гласных: {vowels}, согласных: {len(letters) - vowels}")
        # Мягкий и твёрдый знаки формально не согласные, но для простоты считаем
их к ним.


    else:
        print(f"Тип {type(value).__name__} не поддерживается")

def task2():
    # «Сделать проверку со всеми этими случаями»: вызываем функцию для каждого
типа
    tests = [
        {"яблоко": 5, "груша": 2, "слива": 9},
        [3, 0, 2, 4, 5, 0, 7, 3, 2],
        [1, 2, 3],
        36,
        "А роза упала на лапу Азора",
        "Машинное обучение",
        3.14,
    ]
    for t in tests:
        print("-" * 50)
        analyze(t)
    print("-" * 50)
    s = input("Своя строка для проверки (Enter: пропустить): ")
    if s:
        analyze(s)
```

Задание 3. Сумма каждой строки матрицы

```
def row_sums(matrix):
    """Матрица это список списков, каждая строка сама является списком."""
    return [sum(row) for row in matrix]


def task3():
    try:
        m = int(input("Число строк m: "))
        n = int(input("Число столбцов n: "))
    except ValueError:
        print("Нужны целые числа")
        return
    if m <= 0 or n <= 0:
        print("Размеры должны быть положительными")
        return


    mode = input("Заполнить случайно? (Enter: да, r: ввести вручную):
").strip().lower()
    if mode == "r":
        matrix = []
        for i in range(m):
            row = [int(x) for x in input(f"Строка {i + 1} ({n} чисел через
пробел): ").split()]
            if len(row) != n:
                print(f"В строке должно быть ровно {n} чисел")
                return
            matrix.append(row)
    else:
        # Вложенный генератор: внешний делает m строк, внутренний по n чисел в
строке
```

```
        matrix = [[random.randint(-9, 9) for _ in range(n)] for _ in range(m)]

    print("\nМатрица:")
    for row, s in zip(matrix, row_sums(matrix)):
        print(" ".join(f"{x:4}" for x in row), " | сумма =", s)
```

Задание 4. Демонстрация try/except/finally

```
def safe_divide(a_text, b_text):
    try:
        # Код, в котором может возникнуть ошибка
        a = float(a_text)
        b = float(b_text)
        result = a / b
    except ValueError:
        # Сработает, если float() не смог превратить строку в число
        print(f"  except ValueError: '{a_text}' или '{b_text}' не число")
    except ZeroDivisionError:
        # Сработает при делении на ноль
        print("  except ZeroDivisionError: делить на ноль нельзя")
    else:
        # Выполняется, только если в try ошибок НЕ было
        print(f"  else: {a} / {b} = {result}")
    finally:
        # Выполняется ВСЕГДА: и при ошибке, и без неё.
        # Обычно здесь закрывают файлы и соединения.
        print("  finally: блок finally выполнен")


def task4():
    print("Три сценария: нормальный, деление на ноль, не число\n")
    for a, b in [("10", "4"), ("5", "0"), ("abc", "2")]:
        print(f"safe_divide('{a}', '{b}'):")
        safe_divide(a, b)
    print("\nТеперь сами:")
    safe_divide(input("Делимое: "), input("Делитель: "))
```

Меню выбора задания

```
TASKS = {
    "1": ("Функция prime(n)", task1),
    "2": ("Функция, зависящая от типа аргумента", task2),
    "3": ("Суммы строк матрицы", task3),
    "4": ("try / except / finally", task4),
}


def main():
    while True:
        print("\n===== ЛР1, часть 2, вариант 4 =====")
        for key, (title, _) in TASKS.items():
            print(f"{key}. {title}")
        print("0. Выход")
        choice = input("Номер задания: ").strip()
        if choice == "0":
            break
        if choice in TASKS:
            print(f"\n--- Задание {choice}: {TASKS[choice][0]} ---")
            TASKS[choice][1]()
        else:
            print("Нет такого задания")

if __name__ == "__main__":
    try:
```

```
        main()
    except (EOFError, KeyboardInterrupt):
        print("\nВыход")
```

4.3 Часть 3. Классы и объекты

Задание 1. Класс Square

```
import math

class Square:
    def __init__(self, side):
        # __init__ это конструктор: вызывается при создании объекта Square(5).
        # self это ссылка на сам создаваемый объект, через неё хранятся его
данные.
        if side <= 0:
            raise ValueError("сторона должна быть положительной")
        self.side = side

    def perimeter(self):
        return 4 * self.side

    def area(self):
        return self.side ** 2

    def diagonal(self):
        # По теореме Пифагора: d^2 = a^2 + a^2, значит d = a * sqrt(2)
        return self.side * math.sqrt(2)

def task1():
    try:
        a = float(input("Сторона квадрата: "))
        sq = Square(a)
    except ValueError as e:
        print("Ошибка:", e)
        return
    print(f"Периметр:  {sq.perimeter():.2f}")
    print(f"Площадь:   {sq.area():.2f}")
    print(f"Диагональ: {sq.diagonal():.2f}")
```

Задание 2. Класс Stationery и дочерние классы

```
class Stationery:
    """Родительский (базовый) класс: канцелярская принадлежность."""

    def __init__(self, title):
        self.title = title

    def draw(self):
        print(f"[{self.title}] Запуск отрисовки")

class Pen(Stationery):
    # Pen(Stationery) означает: Pen наследует всё от Stationery (атрибут title,
    # конструктор, метод draw). Ниже мы ПЕРЕОПРЕДЕЛЯЕМ draw своей версией.
    def draw(self):
        print(f"[{self.title}] Ручка: рисуем ровную синюю линию")

class Pencil(Stationery):
```

```
    def draw(self):
        print(f"[{self.title}] Карандаш: делаем набросок, его можно стереть
ластиком")

class Handle(Stationery):
    def draw(self):
        # super() даёт доступ к методу родителя: сначала общее сообщение, потом
своё
        super().draw()
        print(f"[{self.title}] Маркер: выделяем текст жирной жёлтой полосой")

def task2():
    items = [
        Stationery("Принадлежность"),
        Pen("Ручка Parker"),
        Pencil("Карандаш Koh-i-Noor"),
        Handle("Маркер Stabilo"),
    ]
    # Полиморфизм: у всех объектов вызываем один и тот же метод draw(),
    # а Python сам выбирает версию метода по классу объекта.
    for item in items:
        print(f"{type(item).__name__}.draw():")
        item.draw()
        print()
```

Задание 3. Классы Faculty и Student

```
class Faculty:
    def __init__(self, name):
        self.__name = name   # два подчёркивания: «закрытое» поле, доступ через
методы

    def get_name(self):
        return self.__name

    def set_name(self, name):
        if not name.strip():
            raise ValueError("название факультета не может быть пустым")
        self.__name = name

    def __str__(self):
        # __str__ определяет, как объект выглядит в print()
        return self.__name

class Student:
    PASS_MARK = 4        # атрибут класса: общий для всех студентов (10-балльная
шкала)

    def __init__(self, full_name, birth_year, faculty, grades):
        # Поля закрытые. Значения записываем через set-методы, чтобы сразу
        # сработала проверка корректности.
        self.set_full_name(full_name)
        self.set_birth_year(birth_year)
        self.set_faculty(faculty)
        self.set_grades(grades)

    # ---- методы доступа (геттеры) и изменения (сеттеры) ----
    def get_full_name(self):
        return self.__full_name
```

```
    def set_full_name(self, full_name):
        if not full_name.strip():
            raise ValueError("ФИО не может быть пустым")
        self.__full_name = full_name.strip()

    def get_birth_year(self):
        return self.__birth_year

    def set_birth_year(self, year):
        if not 1900 < year < 2020:
            raise ValueError("некорректный год рождения")
        self.__birth_year = year

    def get_faculty(self):
        return self.__faculty

    def set_faculty(self, faculty):
        if not isinstance(faculty, Faculty):
            raise TypeError("факультет должен быть объектом Faculty")
        self.__faculty = faculty

    def get_grades(self):
        return dict(self.__grades)        # отдаём копию, чтобы словарь не
испортили снаружи

    def set_grades(self, grades):
        for subject, mark in grades.items():
            if not 0 <= mark <= 10:
                raise ValueError(f"оценка по «{subject}» должна быть от 0 до
10")
        self.__grades = dict(grades)

    def set_grade(self, subject, mark):
        """Изменить одну оценку (например, после пересдачи)."""
        grades = self.get_grades()
        grades[subject] = mark
        self.set_grades(grades)

    # ---- успеваемость ----
    def average(self):
        return sum(self.__grades.values()) / len(self.__grades) if self.__grades
else 0

    def debts(self):
        return [s for s, m in self.__grades.items() if m < self.PASS_MARK]

    def status(self):
        marks = self.__grades.values()
        if self.debts():
            return "есть задолженности"
        if all(m >= 9 for m in marks):
            return "отличник"
        if all(m >= 6 for m in marks):
            return "хорошист"
        return "успевает"

    def report(self):
        lines = [
```

```
            f"{self.__full_name}, {self.__birth_year} г. р., факультет
{self.__faculty}",
            "  Оценки: " + ", ".join(f"{s} – {m}" for s, m in
self.__grades.items()),
            f"  Средний балл: {self.average():.2f}; статус: {self.status()}",
        ]
        if self.debts():
            lines.append("  Долги по: " + ", ".join(self.debts()))
        return "\n".join(lines)

def read_students():
    """Считывает студентов с клавиатуры. Enter вместо числа: демонстрационные
данные."""
    raw = input("Сколько студентов ввести? (Enter: демо-данные): ").strip()
    if not raw:
        fit = Faculty("ФИТиУ")
        fksis = Faculty("ФКСиС")
        return [
            Student("Иванов Иван Иванович", 2004, fit,
                    {"Математика": 9, "Программирование": 10, "Физика": 9}),
            Student("Петрова Анна Сергеевна", 2005, fksis,
                    {"Математика": 7, "Программирование": 8, "Физика": 6}),
            Student("Сидоров Павел Олегович", 2004, fit,
                    {"Математика": 3, "Программирование": 6, "Физика": 5}),
        ]

    faculties = {}              # один объект Faculty на каждое название
    students = []
    for i in range(int(raw)):
        print(f"\nСтудент {i + 1}")
        name = input("  ФИО: ")
        year = int(input("  Год рождения: "))
        fac_name = input("  Факультет: ").strip()
        faculty = faculties.setdefault(fac_name, Faculty(fac_name))
        grades = {}
        print("  Оценки в формате «предмет оценка», пустая строка: конец")
        while True:
            line = input("    ").strip()
            if not line:
                break
            subject, mark = line.rsplit(maxsplit=1)   # делим по последнему
пробелу
            grades[subject] = int(mark)
        students.append(Student(name, year, faculty, grades))
    return students

def task3():
    try:
        students = read_students()
    except (ValueError, TypeError) as e:
        print("Ошибка ввода:", e)
        return

    print("\n=== Успеваемость ===")
    for st in students:
        print(st.report())

    # Демонстрация методов изменения: студент пересдал экзамен
    debtor = next((st for st in students if st.debts()), None)
    if debtor:
        subject = debtor.debts()[0]
```

```
        debtor.set_grade(subject, 6)
        print(f"\nПосле пересдачи «{subject}» у {debtor.get_full_name()}:")
        print(debtor.report())

    avg = sum(st.average() for st in students) / len(students)
    print(f"\nСредний балл по всем студентам: {avg:.2f}")
```

Задание 4. Собственный класс Temperature

```
class Temperature:
    """Температура. Внутри всегда хранится в градусах Цельсия."""

    count = 0                  # атрибут класса: сколько объектов создано всего
    ABSOLUTE_ZERO = -273.15

    def __init__(self, celsius):
        if celsius < Temperature.ABSOLUTE_ZERO:
            raise ValueError("ниже абсолютного нуля не бывает")
        self.celsius = celsius
        Temperature.count += 1

    # ---- методы экземпляра: первый параметр self, работают с данными объекта -
---
    def to_fahrenheit(self):
        return Temperature.c_to_f(self.celsius)

    def to_kelvin(self):
        return self.celsius - Temperature.ABSOLUTE_ZERO

    def describe(self):
        if self.celsius <= 0:
            return "мороз"
        if self.celsius < 15:
            return "прохладно"
        if self.celsius < 25:
            return "тепло"
        return "жарко"

    def __str__(self):
        return f"{self.celsius:.1f} °C"

    # ---- статический метод: не получает ни self, ни cls.
    # Обычная функция, положенная внутрь класса, потому что по смыслу к нему
относится.
    @staticmethod
    def c_to_f(c):
        return c * 9 / 5 + 32

    @staticmethod
    def f_to_c(f):
        return (f - 32) * 5 / 9

    # ---- методы класса: первый параметр cls, то есть сам класс.
    # Часто служат «альтернативными конструкторами».
    @classmethod
    def from_fahrenheit(cls, f):
        return cls(cls.f_to_c(f))

    @classmethod
```

```
    def from_kelvin(cls, k):
        return cls(k + cls.ABSOLUTE_ZERO)

    @classmethod
    def created(cls):
        return cls.count

def task4():
    t1 = Temperature(22)                    # обычный конструктор
    t2 = Temperature.from_fahrenheit(14)    # метод класса
    t3 = Temperature.from_kelvin(310.15)    # метод класса

    for t in (t1, t2, t3):
        print(f"{t}: {t.to_fahrenheit():.1f} °F, {t.to_kelvin():.2f} K,
{t.describe()}")

    print("\nСтатический метод без создания объекта: 100 °C =",
          Temperature.c_to_f(100), "°F")
    print("Метод класса: создано объектов Temperature:", Temperature.created())

    try:
        Temperature(-300)
    except ValueError as e:
        print("Проверка ошибки: Temperature(-300) ->", e)
```

Меню выбора задания

```
TASKS = {
    "1": ("Класс Square", task1),
    "2": ("Stationery и наследники", task2),
    "3": ("Faculty и Student", task3),
    "4": ("Свой класс Temperature", task4),
}

def main():
    while True:
        print("\n===== ЛР1, часть 3, вариант 4 =====")
        for key, (title, _) in TASKS.items():
            print(f"{key}. {title}")
        print("0. Выход")
        choice = input("Номер задания: ").strip()
        if choice == "0":
            break
        if choice in TASKS:
            print(f"\n--- Задание {choice}: {TASKS[choice][0]} ---")
            TASKS[choice][1]()
        else:
            print("Нет такого задания")

if __name__ == "__main__":
    try:
        main()
    except (EOFError, KeyboardInterrupt):
        print("\nВыход")
```

5 СКРИНШОТЫ ВЫПОЛНЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

5.1 Часть 1. Встроенные типы данных

Результаты выполнения заданий 1 и 2 показаны на рисунке 1. Для числа 12345 сумма цифр равна 15, что подтверждается вторым, арифметическим способом вычисления. Из введённой строки выделены слова-палиндромы «казак», «дед» и «потоп»; однобуквенные слова не учитываются.

```
3. Операции со списком
4. Словарь из двух списков
5. Ювелирный магазин
6. Кортеж из 10 случайных чисел
0. Выход
Номер задания: 1

--- Задание 1: Сумма цифр натурального числа ---
Введите натуральное число: 12345
Сумма цифр числа 12345 = 15
(проверка арифметическим способом: 15)

===== ЛР1, часть 1, вариант 4 =====
1. Сумма цифр натурального числа
2. Слова-палиндромы
3. Операции со списком
4. Словарь из двух списков
5. Ювелирный магазин
6. Кортеж из 10 случайных чисел
0. Выход
Номер задания: 2
На шалаше казак и дед, а потоп ухаживал за коком.
--- Задание 2: Слова-палиндромы ---
Введите строку текста: На шалаше казак и дед, а потоп ухаживал за коком.
Слова-палиндромы: казак, дед, потоп
```

Рисунок 1 – Выполнение заданий 1 и 2

На рисунке 2 приведён результат задания 3 для списка 4, 2, 9, 1, 7, 3, 9. Элементы с нечётными индексами – 2, 1, 3, их произведение равно 6. Наибольший элемент 9 удалён (удаляется первое вхождение), три наибольших элемента исходного списка – 9, 9, 7.

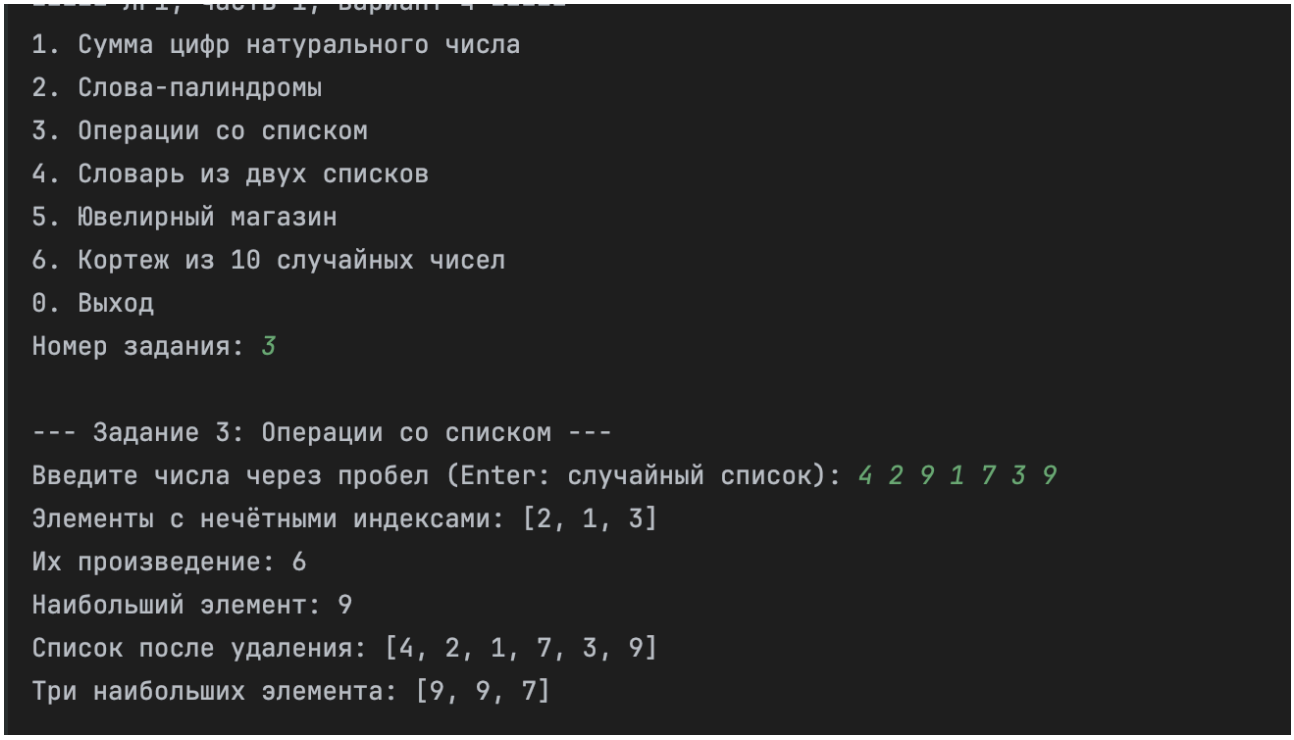


Рисунок 2 – Выполнение задания 3

Результаты заданий 4 и 6 показаны на рисунке 3: из списков a b c и 1 2 3 построен словарь {'a': '1', 'b': '2', 'c': '3'} двумя способами; для кортежа из десяти случайных чисел найдены максимальный и минимальный элементы.

```
Ошибка: длины списков разные (3 и 1)

===== ЛР1, часть 1, вариант 4 =====
1. Сумма цифр натурального числа
2. Слова-палиндромы
3. Операции со списком
4. Словарь из двух списков
5. Ювелирный магазин
6. Кортеж из 10 случайных чисел
0. Выход
Номер задания: 4

--- Задание 4: Словарь из двух списков ---
Первый список (ключи) через пробел: a b c
Второй список (значения) через пробел: 1 2 3
Словарь: {'a': '1', 'b': '2', 'c': '3'}
То же через dict(zip(...)): {'a': '1', 'b': '2', 'c': '3'}
```

Рисунок 3 – Выполнение заданий 4 и 6

5.2 Часть 2. Функции и обработка исключений

На рисунке 4 показана работа функции prime: простое число под номером 10 равно 29; также выведены первые десять простых чисел.

```
/usr/local/bin/python3.8 /Users/fenikhey/Documents/ML/Labs/Lab1/lab1_v4/part2.py

===== ЛР1, часть 2, вариант 4 =====
1. Функция prime(n)
2. Функция, зависящая от типа аргумента
3. Суммы строк матрицы
4. try / except / finally
0. Выход
Номер задания: 1

--- Задание 1: Функция prime(n) ---
Номер простого числа: 10
Простое число №10: 29
Первые 10 простых: [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

===== ЛР1, часть 2, вариант 4 =====
1. Функция prime(n)
```

Рисунок 4 – Выполнение задания 1 (функция prime)

На рисунке 5 показаны суммы строк матрицы 3 × 3 (6, 7 и 7) и демонстрация конструкции try/except/finally: при корректном делении выполняется блок else, при делении на ноль и вводе нечислового значения – соответствующие блоки except; блок finally выполняется во всех случаях.

```
Номер задания: 3

--- Задание 3: Суммы строк матрицы ---
Число строк m: 3
Число столбцов n: 3
Заполнить случайно? (Enter: да, r: ввести вручную): r
Строка 1 (3 чисел через пробел): 1 2 3
Строка 2 (3 чисел через пробел): -4 5 6
Строка 3 (3 чисел через пробел): 0 0 7

Матрица:
   1    2    3  | сумма = 6
  -4    5    6  | сумма = 7
   0    0    7  | сумма = 7

===== ЛР1, часть 2, вариант 4 =====
1. Функция prime(n)
```

Рисунок 5 – Выполнение заданий 3 и 4

5.3 Часть 3. Классы и объекты

Результаты заданий 1 и 2 показаны на рисунке 6. Для квадрата со стороной 5 периметр равен 20, площадь – 25, диагональ – 7,07. Для объектов классов Stationery, Pen, Pencil и Handle метод draw выводит уникальные сообщения; класс Handle дополнительно вызывает метод родителя через super().

```
===== ЛР1, часть 3, вариант 4 =====
1. Класс Square
2. Stationery и наследники
3. Faculty и Student
4. Свой класс Temperature
0. Выход
Номер задания: 1

--- Задание 1: Класс Square ---
Сторона квадрата: 5
Периметр:  20.00
Площадь:   25.00
Диагональ: 7.07

===== ЛР1, часть 3, вариант 4 =====
1. Класс Square
2. Stationery и наследники
```

Рисунок 6 – Выполнение заданий 1 и 2 (классы Square и Stationery)

На рисунке 7 приведён отчёт об успеваемости трёх студентов: для каждого выводятся факультет, оценки, средний балл и статус. У студента с задолженностью по математике после пересдачи (изменение оценки через метод set_grade) статус меняется на «успевает»; средний балл по всем студентам равен 7,33.

```
4. Свой класс Temperature
0. Выход
Номер задания: 3

--- Задание 3: Faculty и Student ---
Сколько студентов ввести? (Enter: демо-данные):

=== Успеваемость ===
Иванов Иван Иванович, 2004 г. р., факультет ФИТиУ
  Оценки: Математика – 9, Программирование – 10, Физика – 9
  Средний балл: 9.33; статус: отличник
Петрова Анна Сергеевна, 2005 г. р., факультет ФКСиС
  Оценки: Математика – 7, Программирование – 8, Физика – 6
  Средний балл: 7.00; статус: хорошист
Сидоров Павел Олегович, 2004 г. р., факультет ФИТиУ
  Оценки: Математика – 3, Программирование – 6, Физика – 5
  Средний балл: 4.67; статус: есть задолженности
  Долги по: Математика

После пересдачи «Математика» у Сидоров Павел Олегович:
Сидоров Павел Олегович, 2004 г. р., факультет ФИТиУ
  Оценки: Математика – 6, Программирование – 6, Физика – 5
  Средний балл: 5.67; статус: успевает

Средний балл по всем студентам: 7.33
```

Рисунок 7 – Выполнение задания 3 (классы Faculty и Student)

Работа класса Temperature показана на рисунке 8: объекты созданы обычным конструктором и методами класса from_fahrenheit и from_kelvin, статический метод c_to_f вызван без создания объекта, метод класса created возвращает число созданных объектов, попытка создать температуру ниже абсолютного нуля обработана как ошибка.

```
===== ЛР1, часть 3, вариант 4 =====
1. Класс Square
2. Stationery и наследники
3. Faculty и Student
4. Свой класс Temperature
0. Выход
Номер задания: 4

--- Задание 4: Свой класс Temperature ---
22.0 °C: 71.6 °F, 295.15 K, тепло
-10.0 °C: 14.0 °F, 263.15 K, мороз
37.0 °C: 98.6 °F, 310.15 K, жарко

Статический метод без создания объекта: 100 °C = 212.0 °F
Метод класса: создано объектов Temperature: 3
Проверка ошибки: Temperature(-300) -> ниже абсолютного нуля не бывает
```

Рисунок 8 – Выполнение задания 4 (класс Temperature)

6 ВЫВОДЫ ПО ЛАБОРАТОРНОЙ РАБОТЕ

В ходе выполнения лабораторной работы были изучены и применены на практике основные встроенные типы данных языка Python. Реализованы программы для обработки чисел и строк с использованием циклов, срезов и строковых методов, выполнены операции над списками, кортежами и словарями; на основе словаря разработано консольное приложение «Ювелирный магазин» с меню, поддерживающее просмотр данных и оформление покупки с проверкой наличия товара и пересчётом остатков.

Освоено создание функций, в том числе функции, изменяющей поведение в зависимости от типа аргумента (isinstance), и функции поиска простого числа по его номеру. Изучена конструкция try/except/else/finally, позволяющая перехватывать ошибки ввода и вычислений (ValueError, ZeroDivisionError) и выполнять завершающие действия независимо от результата.

Получены навыки объектно-ориентированного программирования: созданы классы с методами и конструкторами, реализована инкапсуляция с методами доступа и изменения полей (Faculty, Student), наследование с переопределением методов и полиморфизм (Stationery и дочерние классы), а также методы экземпляра, статические методы и методы класса (Temperature). Результаты выполнения всех заданий совпали с ожидаемыми, что подтверждает правильность разработанных программ.